3F8: Inference: Short Lab Report

Zihe Liu

February 26, 2026

Abstract

We implement a logistic classifier and apply it to a binary classification dataset with non-linearly separable classes. We derive the gradient of the log-likelihood, train the model using gradient ascent, and evaluate performance via log-likelihoods and confusion matrices. We then improve the classifier by expanding the inputs through Gaussian radial basis functions (RBFs) and investigate how the RBF width l governs the bias–variance trade-off. The best-performing model ($l = 0.1$) achieves a test log-likelihood of -0.35, a substantial improvement over the linear baseline (-0.66).

Introduction

Classification is a fundamental task in machine learning: given labelled training data, the goal is to learn a mapping from inputs to discrete class labels that generalises to unseen data. Logistic regression is a simple yet principled approach that models class probabilities via the sigmoid function applied to a linear combination of input features, yielding a linear decision boundary. However, many real-world datasets are not linearly separable.

In this report, we first derive and implement the logistic classifier, then demonstrate its limitations on a non-linearly separable dataset. We subsequently introduce a non-linear feature expansion using Gaussian radial basis functions (RBFs) and investigate how the RBF width parameter l affects model learning and performance.

Exercise a): Gradients of Log-Likelihood

We derive the gradient of the log-likelihood with respect to the parameter vector $\boldsymbol{\beta}$, given a vector of binary labels $\mathbf{y}$ and a matrix of input features $\mathbf{X}$. The probability of a positive class label is given by the logistic (sigmoid) function $\sigma(a) = 1/(1 + e^{-a})$ applied to the activations:

$$P(y^{(n)} = 1 \mid \tilde{\mathbf{x}}^{(n)}) = \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}), \qquad P(y^{(n)} = 0 \mid \tilde{\mathbf{x}}^{(n)}) = 1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})$$

where $\tilde{\mathbf{x}}^{(n)} = (1, \mathbf{x}^{(n)})^T$ are the bias-augmented inputs, so that $\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)} = \beta_0 + \sum_{d=1}^{D} \beta_d x_d^{(n)}$. These two cases can be written compactly as a single Bernoulli likelihood:

$$P(y^{(n)} \mid \tilde{\mathbf{x}}^{(n)}, \boldsymbol{\beta}) = \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})^{y^{(n)}} \left(1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})\right)^{1 - y^{(n)}}$$

Assuming the N datapoints are independent and identically distributed, the dataset log-likelihood is:

$$\mathcal{L}(\boldsymbol{\beta}) = \log P(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}) = \sum_{n=1}^{N} \left[y^{(n)} \log \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}) + (1 - y^{(n)}) \log \left(1 - \sigma(\boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)})\right) \right]$$

Let $a^{(n)} = \boldsymbol{\beta}^T \tilde{\mathbf{x}}^{(n)}$, so that $\frac{\partial a^{(n)}}{\partial \boldsymbol{\beta}} = \tilde{\mathbf{x}}^{(n)}$. Noting the derivative of the logistic function $\frac{d\sigma(a)}{da} = \sigma(a)\,(1 - \sigma(a))$,

$$\frac{\partial}{\partial \boldsymbol{\beta}} \left[y^{(n)} \log \sigma(a^{(n)}) \right] = y^{(n)} \cdot \frac{1}{\sigma(a^{(n)})} \cdot \underbrace{\frac{d\sigma}{da}}_{\sigma(1-\sigma)} \cdot \underbrace{\frac{\partial a^{(n)}}{\partial \boldsymbol{\beta}}}_{\tilde{\mathbf{x}}^{(n)}} = y^{(n)} \left(1 - \sigma(a^{(n)})\right) \tilde{\mathbf{x}}^{(n)}$$

$$\frac{\partial}{\partial \boldsymbol{\beta}}\left[(1-y^{(n)})\log(1-\sigma(a^{(n)}))\right] = (1-y^{(n)})\cdot\frac{-\sigma(a^{(n)})(1-\sigma(a^{(n)}))}{1-\sigma(a^{(n)})}\cdot\tilde{\mathbf{x}}^{(n)} = -(1-y^{(n)})\sigma(a^{(n)})\tilde{\mathbf{x}}^{(n)}$$

Combining these two contributions for datapoint n:

$$y^{(n)}(1-\sigma(a^{(n)}))\tilde{\mathbf{x}}^{(n)} - (1-y^{(n)})\sigma(a^{(n)})\tilde{\mathbf{x}}^{(n)} = \left(y^{(n)} - \sigma(a^{(n)})\right)\tilde{\mathbf{x}}^{(n)}$$

Summing over all N datapoints,

$$\boxed{\frac{\partial \mathcal{L}(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = \sum_{n=1}^{N}\left(y^{(n)} - \sigma(\boldsymbol{\beta}^T\tilde{\mathbf{x}}^{(n)})\right)\tilde{\mathbf{x}}^{(n)} = \tilde{\mathbf{X}}^T(\mathbf{y}-\boldsymbol{\sigma})}$$

where $\tilde{\mathbf{X}}$ is the $N\times(D+1)$ matrix whose n-th row is $(\tilde{\mathbf{x}}^{(n)})^T$, and $\boldsymbol{\sigma}$ is the N-dimensional vector with entries $\sigma(\boldsymbol{\beta}^T\tilde{\mathbf{x}}^{(n)})$. The vectorised form follows because the sum $\sum_n(\cdot)\tilde{\mathbf{x}}^{(n)}$ is equivalent to the matrix product $\tilde{\mathbf{X}}^T$ applied to the residual vector $(\mathbf{y}-\boldsymbol{\sigma})$.

Exercise b) Vectorized Gradient Ascent

In this exercise we write vectorised pseudocode to estimate the parameters $\boldsymbol{\beta}$ using gradient ascent of the log-likelihood. The update rule is

$$\boldsymbol{\beta}^{(\text{new})} = \boldsymbol{\beta}^{(\text{old})} + \eta\left.\frac{\partial \mathcal{L}(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}}\right|_{\boldsymbol{\beta}=\boldsymbol{\beta}^{(\text{old})}} = \boldsymbol{\beta}^{(\text{old})} + \eta\,\tilde{\mathbf{X}}^T\left(\mathbf{y}-\boldsymbol{\sigma}^{(\text{old})}\right),$$

where $\boldsymbol{\sigma}^{(\text{old})}$ is the vector of predictions $\sigma(\boldsymbol{\beta}^{(\text{old})T}\tilde{\mathbf{x}}^{(n)})$ under the current parameters. The pseudocode is as follows:

```
Function estimate_parameters:

   Input:  feature matrix X (N x D), labels y (N x 1),
           learning rate eta, number of iterations T
   Output: vector of coefficients b ((D+1) x 1)

   Code:
      X_tilde = [ones(N,1), X]        % augment with bias column
      b = randn(D+1, 1)               % initialise randomly

      for t = 1 to T:
          sigma = 1 / (1 + exp(-X_tilde * b))   % predictions (N x 1)
          grad = X_tilde^T * (y - sigma)         % gradient ((D+1) x 1)
          b = b + eta * grad                     % gradient ascent step

      return b
```

The learning rate η should be chosen small enough that the log-likelihood increases roughly monotonically, but large enough that convergence is achieved within a reasonable number of iterations. The number of iterations T should be large enough for the log-likelihood to plateau.

Exercise c) Data Visualisation

The dataset consists of $N = 1000$ datapoints with two-dimensional input features $\mathbf{x}^{(n)} \in \mathbb{R}^2$ and binary class labels $y^{(n)} \in \{0, 1\}$. From Figure 1, we observe that Class 2 (blue) is concentrated in the upper and lower area of the input space, while Class 1 (red) is interspersed between the 2 blue regions. The two classes overlap significantly, particularly around the boundaries, and no single straight line can cleanly separate them. A linear classifier produces a decision boundary of the form $\boldsymbol{\beta}^T\tilde{\mathbf{x}} = 0$, which is a straight line in 2D. Given the non-linear arrangement of the classes, such a linear boundary would inevitably misclassify a substantial portion of the data. A non-linear decision boundary would be needed to capture the true class structure more accurately.

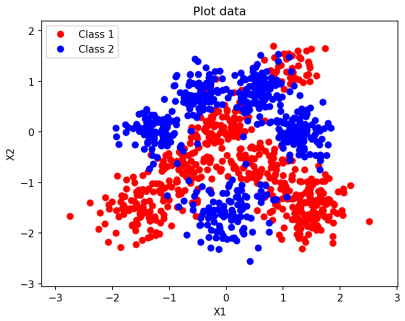

Figure 1: Visualisation of the dataset in the (X_1, X_2) input space. Red points are Class 1 ($y = 0$) and blue points are Class 2 ($y = 1$).

Exercise d) Training the Linear Classifier

The data is split randomly[1] into training ($N_{\text{train}} = 800$) and test ($N_{\text{test}} = 200$) sets. Gradient ascent is implemented as:

```
sigmoid_value = predict(X_tilde_train, w)
w = w + alpha * np.dot(X_tilde_train.T, y_train - sigmoid_value)
```

We train the classifier with learning rate $\eta = 0.0005$ for $T = 100$ iterations. The average log-likelihood on the training and test sets as the optimisation proceeds is shown in Figure 2.

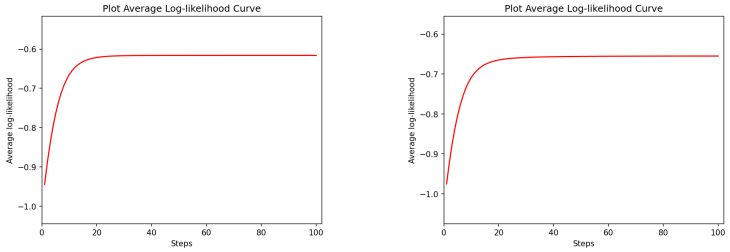

Figure 2: Learning curves showing the average log-likelihood per datapoint on the training (left) and test (right) datasets.

Both curves increase monotonically and plateau after approximately 20 iterations, indicating that the optimisation has converged and $\eta = 0.0005$ is an appropriate learning rate. The training and test curves follow a similar trajectory, suggesting there is no overfitting – as expected for a model with only $D + 1 = 3$ parameters fitting 800 datapoints.

Figure 3 displays the contours of the predictive probabilities overlaid on the data. The contours are parallel straight lines, confirming the linear nature of the decision boundary. The 0.5 contour (the decision boundary) runs roughly diagonally, separating the upper region (higher $P(y = 1)$) from the lower region. However, as anticipated in Exercise c), a linear boundary cannot capture the non-linear class structure: many red (Class 1) and blue (Class 2) points lie on the wrong side of the decision boundary.

Exercise e) Log-Likelihoods and Confusion Matrix

The final average training and test log-likelihoods are shown in Table 1. The test log-likelihood (-0.6551) is slightly lower than the training log-likelihood (-0.6161), which is expected since the model was fitted to the training data. The small gap between them confirms the absence of significant overfitting. However both values are close to $-\ln 2 \approx -0.693$, which is the log-likelihood of a random coin-flip classifier, indicating

[1] A fixed numpy random seed of 1 was used for reproducibility.

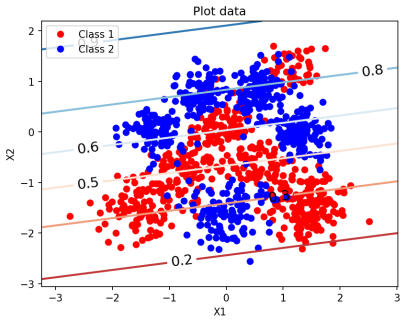

Figure 3: Contours of the class predictive probabilities for the linear logistic classifier.

that the linear model offers only a modest improvement over random guessing on this non-linearly separable dataset.

The 2×2 confusion matrix on the test set (using threshold $\tau = 0.5$) is shown in Table 2. The classifier correctly identifies 71.88% of Class 1 ($y = 1$) points and 61.54% of Class 0 ($y = 0$) points. However, the error rates are substantial: 38.46% of true Class 0 points are misclassified as Class 1, and 28.12% of true Class 1 points are misclassified as Class 0. This confirms that the linear decision boundary is inadequate for this dataset.

Avg. Train ll	**Avg. Test ll**
-0.6161	-0.6551

Table 1: Average training and test log-likelihoods for the linear classifier.

		$\hat{y}$	
		0	1
y	0	0.6154	0.3846
	1	0.2812	0.7188

Table 2: Confusion matrix on the test set (fractions conditioned on true label).

Exercise f) Non-Linear Feature Expansion with RBFs

To overcome the limitations of the linear classifier, we expand the original two-dimensional inputs through a set of N_{train} Gaussian radial basis functions (RBFs), each centred on one training datapoint:

$$\tilde{\mathbf{x}}_{\text{RBF}}^{(n)} = \left(1,\ \phi_1(\mathbf{x}^{(n)}),\ \phi_2(\mathbf{x}^{(n)}),\ \ldots,\ \phi_{N_{\text{train}}}(\mathbf{x}^{(n)})\right)^T \in \mathbb{R}^{N_{\text{train}}+1}$$

where each basis function ϕ_m is a Gaussian kernel centred on the m-th training point $\mathbf{x}^{(m)}$:

$$\phi_m(\mathbf{x}^{(n)}) = \exp\left(-\frac{1}{2l^2}\sum_{d=1}^{D}\left(x_d^{(n)} - x_d^{(m)}\right)^2\right) = \exp\left(-\frac{\|\mathbf{x}^{(n)} - \mathbf{x}^{(m)}\|^2}{2l^2}\right)$$

The hyperparameter $l > 0$ is the *width* of the RBFs. Intuitively, $\phi_m(\mathbf{x}^{(n)})$ measures how close $\mathbf{x}^{(n)}$ is to the m-th training centre: it equals 1 when $\mathbf{x}^{(n)} = \mathbf{x}^{(m)}$ and decays to 0 as the Euclidean distance $\|\mathbf{x}^{(n)} - \mathbf{x}^{(m)}\|$ grows relative to l. The logistic classifier is now applied in this $(N_{\text{train}} + 1)$-dimensional feature space rather than the original $(D + 1)$-dimensional space.

Experimental Results

We train the logistic classifier with RBF features for three widths $l \in \{0.01, 0.1, 1\}$, using learning rates $\eta \in \{0.0001, 0.001, 0.00005\}$ and $T = 500$ gradient ascent steps for each, respectively. The learning rates were chosen so that the log-likelihood increases roughly monotonically without oscillation; Figure 4 displays our results.

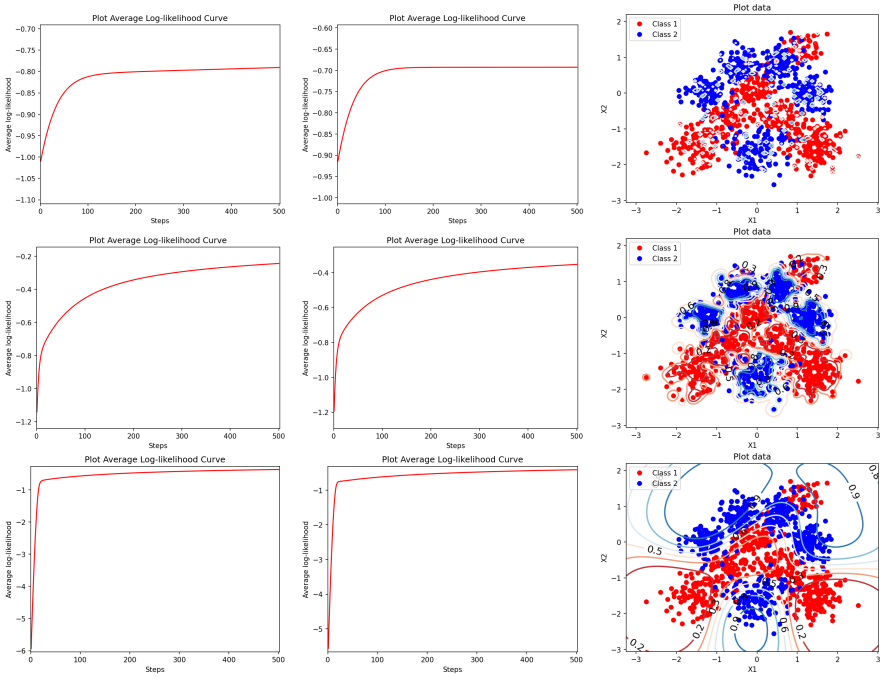

Figure 4: Train log-likelihood (left), test log-likelihood (middle), and predictive contours (right). **Top:** $l = 0.01$, $\eta = 0.0001$. **Mid:** $l = 0.1$, $\eta = 0.001$. **Bot:** $l = 1.0$, $\eta = 0.00005$. All $T = 500$.

Discussion of results

Role of the width l

The width l controls the *smoothness* of the feature expansion and thereby governs the bias–variance trade-off:

- **Small l (e.g. $l = 0.01$):** Each ϕ_m is extremely narrow – essentially non-zero only at its own centre. The feature matrix approximates the identity, so the model cannot interpolate between training points (generalize beyond train set). The contours confirm this: predictions are highly localised spikes, and both test and train log-likelihoods are near $-\ln 2 \approx -0.693$, basically random guessing. Overfitting has clearly occurred.
- **Intermediate l (e.g. $l = 0.1$):** Basis functions overlap enough for neighbouring points to share information, yet remain localised enough to capture non-linear structure. The contours show structured decision regions matching the true class distribution. Train/test log-likelihoods ($-0.2433/-0.3537$) are a substantial improvement over both the linear classifier and $l = 0.01$.
- **Large l (e.g. $l = 1$):** Broad basis functions produce simpler (closer to linear) decision areas, where the contours are smooth and capture large-scale class structure well. Train/test log-likelihoods ($-0.3676/-0.4143$) show a small gap, indicating good generalisation. If l is too large the model underfits.

Overall, increasing l from 0.01 to 1.0 transitions the model from overly localised (overfitting to training data) through well-structured non-linear boundaries to smooth, close to linear decision boundaries.

Impact on the learning rate η

For small l, the feature matrix approximates the identity with small gradient magnitudes, so a moderate η suffices. For intermediate l, features overlap and gradient magnitudes grow, so a larger η speeds convergence. For large l, features are highly correlated with large gradient magnitudes, demanding a smaller η to avoid divergence. Figure 5 demonstrates this: using $\eta = 0.0005$ ($10\times$ larger) for $l = 1.0$ causes severe oscillations as the gradient steps overshoot the optimum each iteration.

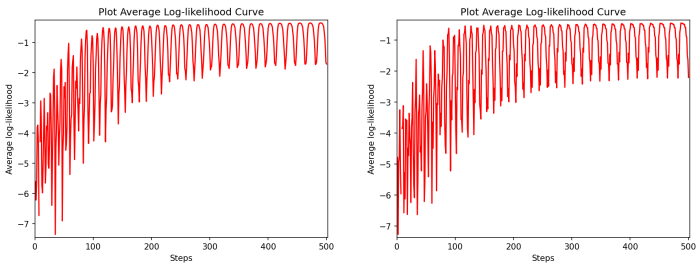

Figure 5: Train (left) and test (right) log-likelihood for $l = 1.0$ with $\eta = 0.0005$, demonstrating oscillations from an excessively large learning rate.

Exercise g) Comparison of Results

The final training and test log-likelihoods per datapoint for all four models (linear and RBF with $l \in \{0.01, 0.1, 1\}$) are summarised in Table 3. The corresponding confusion matrices on the test set (threshold $\tau = 0.5$) are shown in Tables 4–7.

Model	**Avg. Train ll**	**Avg. Test ll**
Linear	-0.6161	-0.6551
RBF $l = 0.01$	-0.7907	-0.6927
RBF $l = 0.1$	-0.2433	-0.3537
RBF $l = 1.0$	-0.3676	-0.4143

Table 3: Average log-likelihoods per datapoint for all models.

		$\hat{y}$	
		0	1
y	0	0.6154	0.3846
	1	0.2812	0.7188

Table 4: Conf. matrix, linear.

		$\hat{y}$	
		0	1
y	0	0.9615	0.0385
	1	0.9167	0.0833

Table 5: Conf. matrix, $l = 0.01$.

		$\hat{y}$	
		0	1
y	0	0.9327	0.0673
	1	0.2083	0.7917

Table 6: Conf. matrix, $l = 0.1$.

		$\hat{y}$	
		0	1
y	0	0.7885	0.2115
	1	0.1250	0.8750

Table 7: Conf. matrix, $l = 1$.

The $l = 0.1$ model achieves the best test log-likelihood (-0.3537 vs. -0.6551 linear) with both high true negative (0.93) and true positive (0.79) rates, reflecting its ability to capture non-linear class structure. The $l = 1.0$ model has the best true positive rate (0.875) but a higher false positive rate (0.21), as its smooth features lack resolution for finer boundaries. The $l = 0.01$ model performs near random guessing (test ll -0.6927), with a confusion matrix biased toward $\hat{y} = 0$ (true positive rate only 0.08) because the narrow basis functions mean test points almost always lie outside any decision boundaries - the model never predicts confidently for any class. Compared to the linear classifier, all RBF models reduce the false positive rate (0.38 for linear). However, only $l = 0.1$ and $l = 1.0$ maintain a high true positive rate, confirming that an appropriately chosen RBF width substantially improves classification.

Conclusions

The linear logistic classifier achieves a test log-likelihood of -0.6551, only marginally better than random guessing ($-\ln 2 \approx -0.693$), confirming its inability to model the non-linear class structure. Expanding the inputs with Gaussian RBFs yields substantial improvements: $l = 0.1$ achieves the best test log-likelihood (-0.3537) with balanced classification accuracy, while $l = 1.0$ achieves the highest true positive rate at the cost of more false positives. The RBF width l controls the bias–variance trade-off: too small gives overly localised features that fail to generalise, while too large smooths away fine-grained class structure. Limitations include the lack of regularisation (which could mitigate overfitting for small l), and that the learning rates were chosen by manual tuning rather than a systematic search.